

How it works.

A layered guide to what DataBridge does, how users experience it, and what happens under the hood.

Part I — At a glance one-page summary

Part II — The user journey for business users & admins

Part III — Under the hood for data engineers & technical readers

Part IV — Architecture for developers joining the codebase

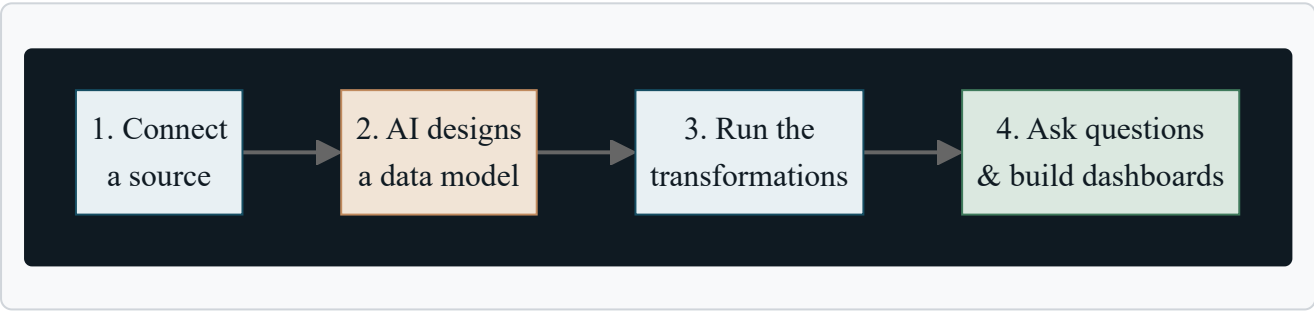
Part I — At a glance

AUDIENCE

Anyone. If you only read one section, read this one.

DataBridge is an AI-native semantic data platform. Business users connect their operational databases (their ERP, CRM, whatever they already run), let the AI understand the data, and then ask questions in plain English or build dashboards by describing them. The platform turns natural language into SQL, executes it against a purpose-built analytics warehouse, and renders the answer. No SQL expertise required to use it, no data team required to set it up.

From a user's perspective, there are four moments:



Three kinds of users experience the platform differently:

ROLE	SEES	DOES
Admin	Connection setup, semantic model editor, transformation runs, team management	One-time setup; reviews AI-drafted definitions; approves the schema the AI designs
Analyst	Chat, dashboards, notebooks, star-schema diagrams	Asks questions, builds dashboards, designs data products alongside the admin
Viewer	Chat, dashboards, reports	Consumes — asks questions, views dashboards; can't edit the model

The platform is **multi-tenant** (each company's data is isolated from every other company's) and the AI is **Claude from Anthropic**, used at every stage where understanding or generation of SQL is needed.

Part II — The user journey

AUDIENCE

Business users and admins. No code, no infrastructure talk.

1. Connecting a source

An admin opens **Sources**, provides connection details for the client's operational database (SQLite, PostgreSQL, MySQL, or SQL Server), and DataBridge tests the connection. Credentials are encrypted at rest with AES-256-GCM before being stored.

Once connected, the admin clicks **Profile**. In the background, DataBridge reads every table and column in the source, runs statistical analysis (null rates, distinct values, minimum/maximum), then asks Claude to generate plain-English descriptions of every table and column. The admin gets a ready-to-review *data dictionary*, usually within a minute or two.

The admin then opens **Semantics** to read through the AI's draft, correct anything that's wrong, and confirm. This is the one-time upfront cost. After this, the platform knows what the data *means* — which is the foundation everything else rests on.

2. Designing a data model

An admin (or analyst) creates a **data product** — a coherent slice of the business like "Sales", "Customers", or "Inventory". The admin types a short description. The AI reads the source schema + the confirmed semantic definitions, then designs a *Kimball star schema*: fact tables (e.g. sales transactions) surrounded by dimension tables (customers, products, dates). It also writes the SQL to transform raw source data into these tables.

The admin reviews the proposed schema on a visual diagram (**ReactFlow**-rendered), tweaks descriptions or column roles if needed, and approves. The AI drafts the schema; the human confirms it — never the other way around.

3. Running the transformations

The admin clicks **Run transformation**. DataBridge takes the approved SQL for each table, executes it against the source data, and writes the results as Parquet files (a fast, compressed analytical format) into a warehouse. This happens either on local disk (for dev) or in Azure Blob Storage (for production). After each table materialises, automated **quality gates** run — checking for things like duplicate business keys or JOINS that silently multiply rows. Failures block the run and surface as specific error messages in the UI.

The admin can set a schedule (e.g. "rebuild every night at 3 am") so the data products stay fresh without manual intervention.

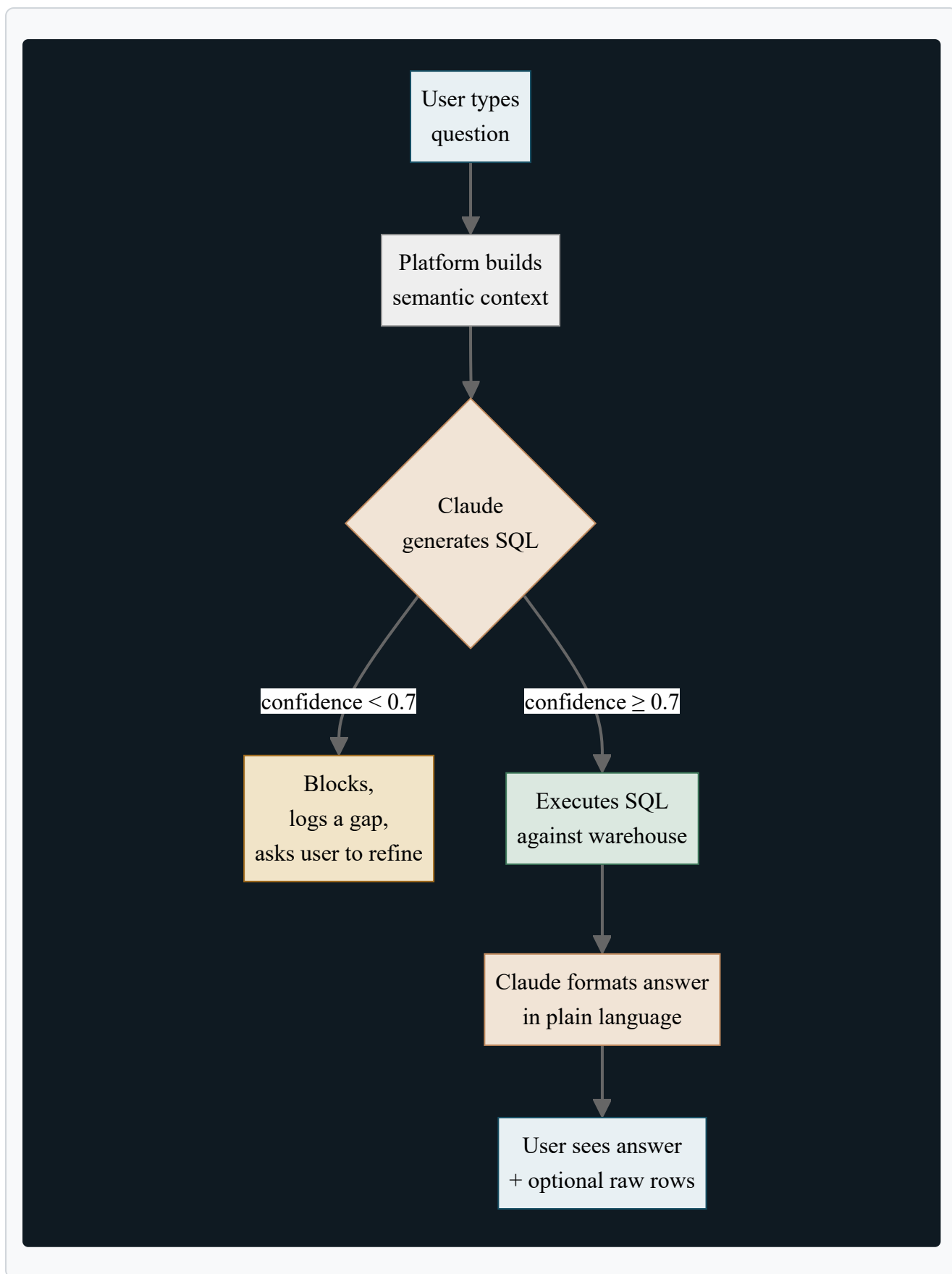
4. Asking questions and building dashboards

Any user — viewer, analyst, admin — opens **Ask** and types a question in plain English:

"Who are our top 10 customers by revenue in 2025, and how does that compare to last year?"

Behind the scenes, the platform turns that into SQL, runs it, and responds with both a natural-language answer and the underlying data. If the AI isn't confident enough, it says so rather than guessing — and quietly logs the question as a gap to improve.

The **Dashboards** builder works similarly: describe what you want to see ("sales overview with monthly trend and top products"), answer a few clarifying questions the AI asks, and it generates a full interactive dashboard. **Notebooks** provide a Python + SQL editor in the browser for power users.



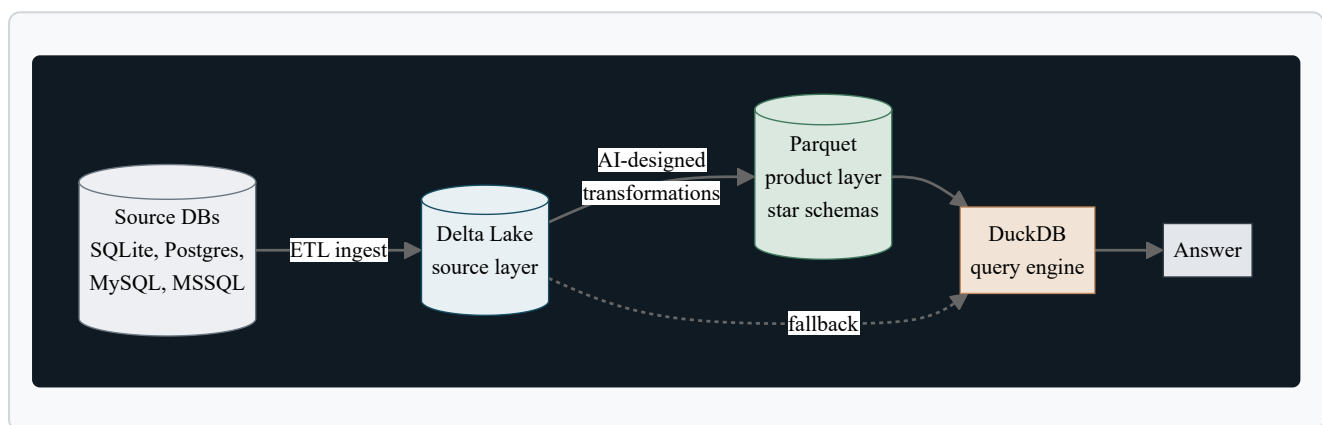
Part III — Under the hood

AUDIENCE

Data engineers and technical readers. Some SQL + distributed systems assumed.

The two-layer data model

DataBridge maintains data at two layers. The **source layer** is a faithful ingested copy of the client's operational database, stored as Delta Lake tables. The **product layer** is the materialised Kimball star schemas, stored as Parquet files. Queries prefer the product layer when it exists (cleaner model, purpose-built for analytics) and fall back to the source layer when a product doesn't cover the question.



Ingestion: source databases to Delta Lake

Ingestion runs as a separate Python FastAPI service (the **ETL service**). When the backend triggers an ingest, the ETL reads table data from the source connector (it supports all 4 source types), converts to Apache Arrow, and writes Delta Lake tables via the `deltalake` library. Delta Lake gives us ACID transactions, schema evolution, and time-travel — modern table format essentials, with a `_delta_log/` directory per table storing the transaction history.

Ingests can be full (overwrite) or incremental (watermark-based append). For incremental, the ETL tracks a high-water-mark column per table (e.g. `updated_at`) and only loads rows past the previous run's max.

Transformations: Delta Lake to Parquet star schemas

Star-schema materialisation runs through **dbt-duckdb**. When an admin clicks *Run*, the backend generates a complete dbt project on disk — one SQL file per table, a YAML file declaring sources, and custom test macros for quality gates. It POSTs to the ETL service which shells out to `dbt run` followed by `dbt test`. dbt figures out the DAG order from `{{ ref() }}` macros in the SQL, builds dimensions before facts, runs the tests, and exports the results as Parquet files.

If the source schema already covers what analysts need, the product layer is optional — queries fall back to reading Delta tables directly via DuckDB's `delta_scan()` function.

Querying: DuckDB reads Parquet

All analytical queries run through **DuckDB** — an embedded OLAP engine. The backend maintains a pool of in-memory DuckDB instances, each with views pre-registered over the relevant Delta/Parquet files. When a query comes in, the pool serves it a ready-to-execute instance; the instance is reused across concurrent queries. Pools invalidate automatically when an ingest or transformation writes new data.

Reads are cheap because Parquet is column-oriented — for a query like "sum of revenue by month", DuckDB reads only the `revenue` + `date` columns, skipping everything else.

Natural language to SQL

When a user types a question, the platform:

1. Extracts referenced entities from the question (e.g. "customers", "orders")
2. Retrieves relevant table + column definitions from the semantic graph
3. Sends question + context + past conversation history to Claude
4. Claude returns SQL, a confidence score (0–1), and the list of tables it used
5. If confidence ≥ 0.7 , the platform executes the SQL; otherwise it blocks, logs a *definition gap*, and asks the user to refine
6. On result, Claude is called a second time to format a plain-language answer

Repeated questions hit a **query cache**: the hash of (question + context + domain filters) is looked up first; on match, the cached SQL + confidence is reused. This typically serves cache-hit questions in under 100 ms instead of the 2–5 s a fresh Claude call takes.

Definition gaps: when the AI doesn't know enough

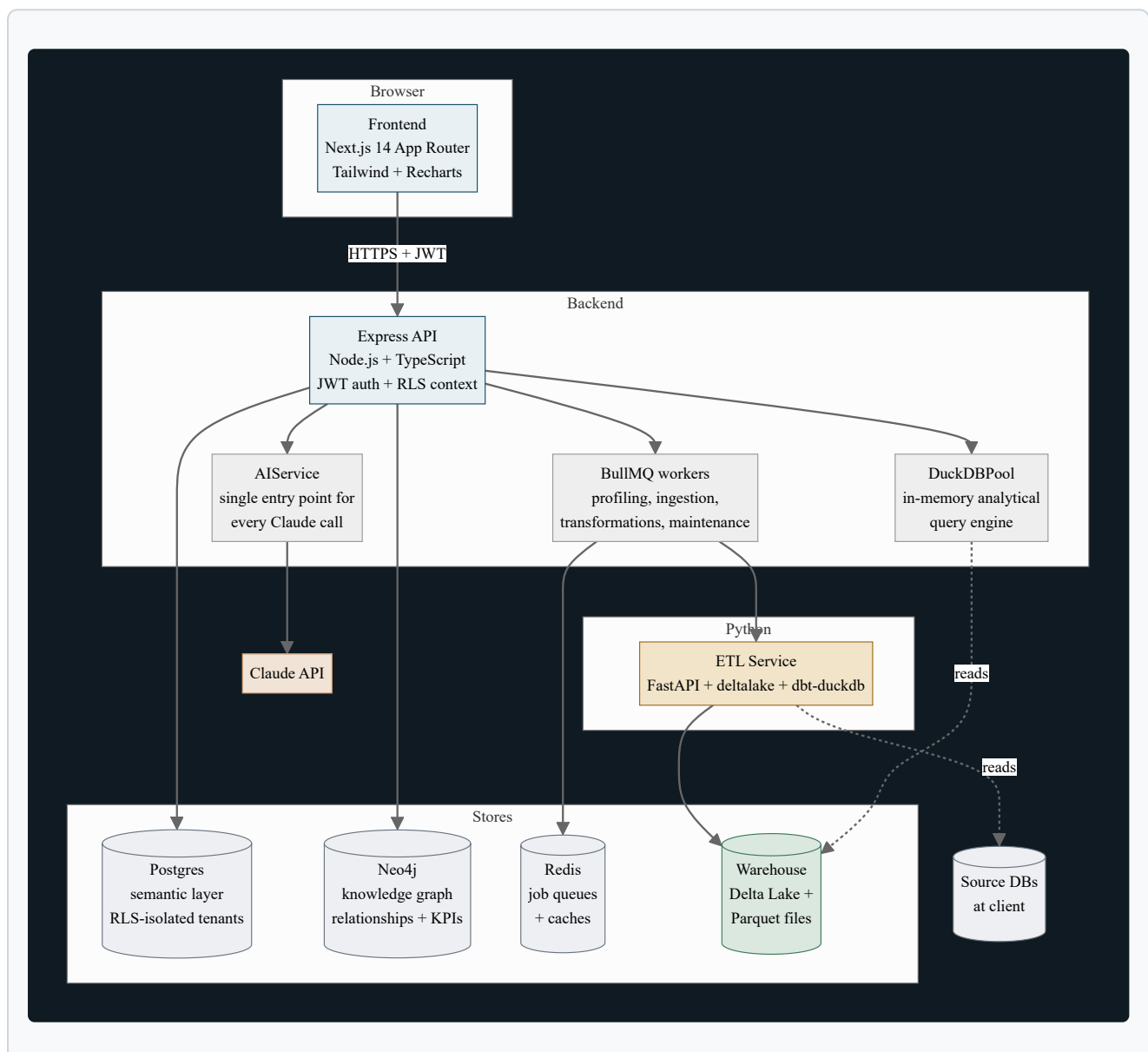
If Claude's confidence is below threshold, the system doesn't guess. It surfaces a polite *I don't have enough context for this yet* message and silently logs a **definition gap** to the admin. The gaps page shows questions the platform couldn't answer and points admins at which definitions to improve. Over time the gaps list shrinks as the semantic model tightens.

Part IV — Architecture

AUDIENCE

Developers joining the codebase or reasoning about deployment.

Component overview



Multi-tenancy

Every row in every Postgres table carries a `tenant_id`. PostgreSQL's **Row-Level Security (RLS)** policies enforce that queries only see rows matching the caller's tenant. The backend sets `SET app.current_tenant = <id>` on each request from the JWT, and uses a non-superuser role

(databridge_app) so the policies are enforced. A migrations role (databridge) bypasses RLS for schema changes.

The warehouse is tenant-isolated by path convention: warehouse/_dbt_state/tenant_<id>/... , az://warehouse/tenant_<id>/... . Tenant boundaries carry through to background jobs via job-data payloads.

AI service: one call site, one SDK

Every Claude API call in the backend goes through backend/src/ai/AIService.ts . Prompts live next to it in ai/prompts/ , one file per call type:

CALL TYPE	PURPOSE
schemaDraftPrompt	AI describes every table / column during profiling
n1ToSqlPrompt	Natural language → SQL for source-layer questions
n1ToSqlPromptDuckDB	Same for product-layer queries (DuckDB dialect)
starSchemaPrompt	Designs star schema + transformation SQL, per data product
dashboardPrompt	Generates dashboard specs (widgets + filters + SQL)
answerFormatterPrompt	Summarises a query result in 1–3 business sentences
repairPrompt	Agentic SQL repair loop when a result looks suspicious

Centralising every Claude call means token counts, latency, retries, and cost tracking are all measured in one place.

Background jobs

Long-running work uses **BullMQ** queues backed by Redis. If Redis isn't configured, jobs fall back to inline execution (synchronous). Four queues exist:

- schema-profiling — kicked off when an admin connects a source
- ingestion — triggered manually or on a schedule; shells out to the ETL service
- transformation — builds a data product's star schema (now via dbt)
- warehouse-maintenance — weekly Delta OPTIMIZE + VACUUM to keep file counts bounded and query times stable

Observability

When APPLICATIONINSIGHTS_CONNECTION_STRING is set, structured metrics flow into Azure Application Insights:

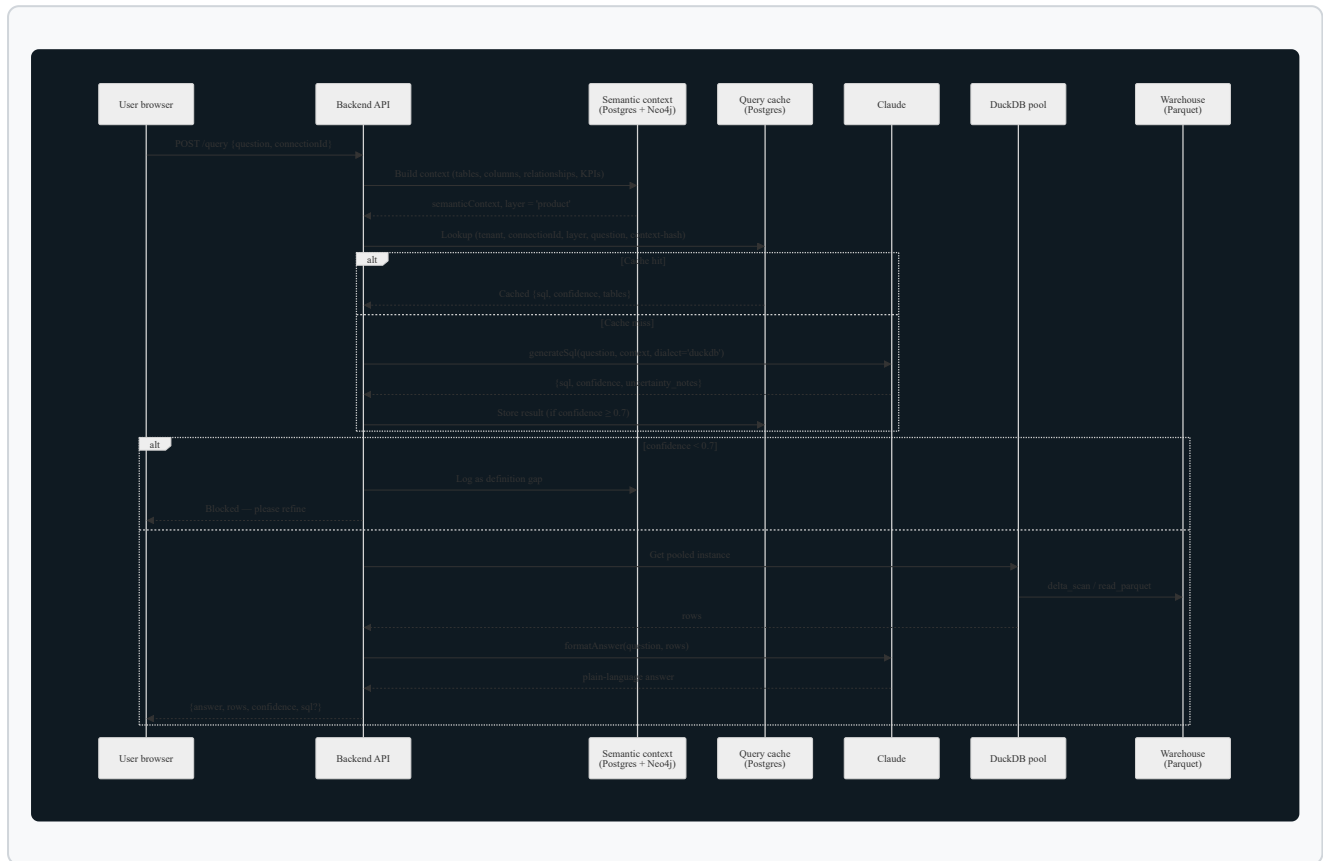
- n1_to_sql_ms — NL→SQL latency, tagged with layer (product/source) and cache (hit/miss)
- duckdb_query_ms — analytical query execution latency

- `ingestion_ms` — ETL duration per run
- `transformation_ms` — product-materialisation duration
- `ai_input_tokens` / `ai_output_tokens` — per-call Claude token counts

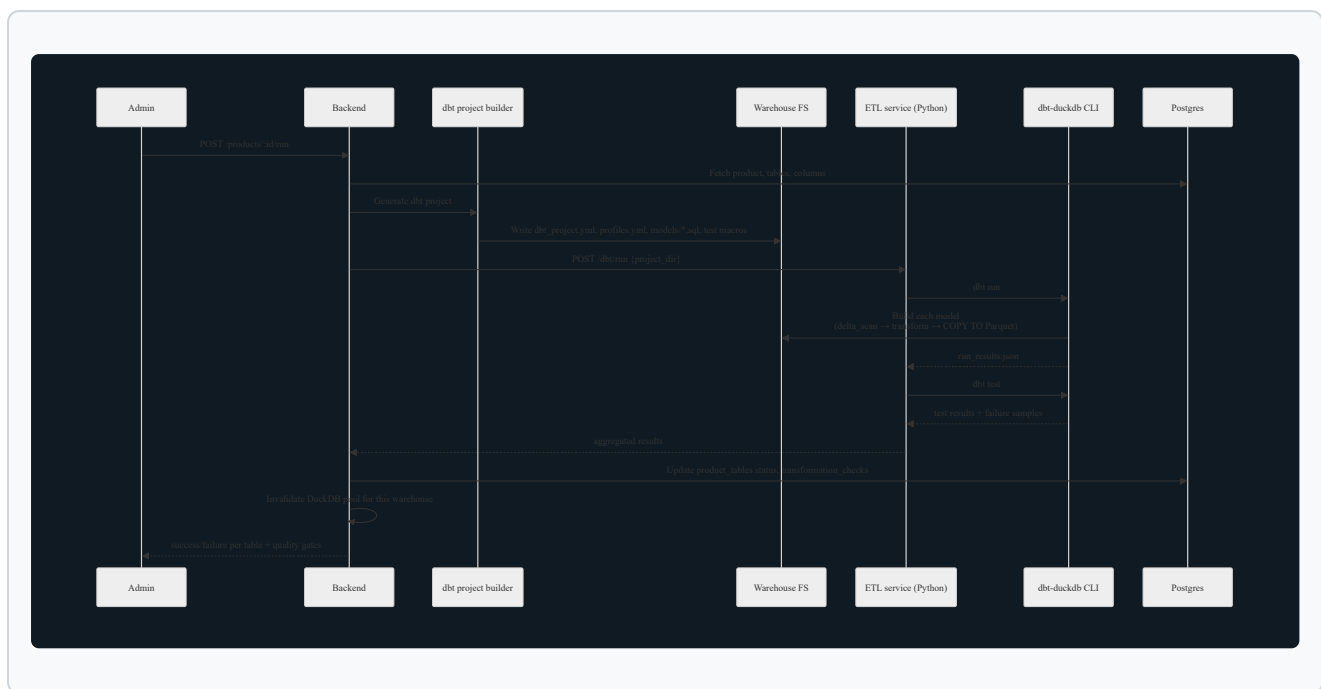
When App Insights isn't configured, all of these silently no-op — it's optional infrastructure.

Part V — Flow deep-dives

A single query, end to end



A product transformation, end to end



Per-tenant path convention

Everything that lives on disk (or Blob) is partitioned by tenant:

```

warehouse/
├── conn_22/                                ← ingested source layer
│   ├── customers/_delta_log/...
│   └── orders/_delta_log/...
├── product/                                ← product layer
│   └── articles/
│       ├── dim_article/data.parquet
│       └── fact_sales/data.parquet
├── _dbt_projects/tenant_63/articles/        ← generated dbt project
│   ├── dbt_project.yml
│   ├── profiles.yml
│   └── models/
└── _dbt_state/tenant_63/articles/          ← persistent DuckDB state for incremental
    └── state.duckdb
  
```

Part VI — Recent architecture work

AUDIENCE

Dev team reference for what changed in April 2026.

Infrastructure hardening (6 focused phases)

#	CHANGE	WIN
1	Persistent DuckDB pool with pre-registered views + invalidation on ingestion/transformation	Analytical queries after the first skip the DuckDB spin-up cost; enables query caching
2	Delta Lake <code>mergeSchema=True</code> on incremental appends + weekly <code>OPTIMIZE</code> / <code>VACUUM</code> scheduled job	Source schema can evolve without manual migrations; small-file accumulation never kills query performance
3	Removed regex-based SQL auto-fix in transformation runner	Silent wrong fixes eliminated; errors surface DuckDB's own candidate-binding suggestions
4	NL→SQL query cache keyed on (tenant, connection, layer, question, context-hash) with 24h TTL	Repeat questions serve in <100 ms instead of 2–5 s; cuts dominant Claude API cost
5	Observability metrics across NL→SQL, DuckDB execution, ETL, transformations, Claude tokens	p95 latencies and error rates measurable per-layer; cache hit rate is a dashboard metric
6	Two design RFCs filed: dbt migration, and Python kernel for notebooks	Big architectural bets have written context, not tribal knowledge

dbt migration (RFC-001 shipped)

The homegrown `transformationRunner.ts` (620 lines) was replaced by a dbt-duckdb integration. A feature flag `USE_DBT_TRANSFORMATIONS` routes between the two engines; both coexist while the rollout is validated. The new engine gains:

- **Real DAG resolution** from `{{ ref() }}` macros in model SQL — dbt figures out build order, we don't
- **Incremental loads** via `materialized='incremental'` + `unique_key` + delete+insert strategy, with a persistent DuckDB state file that survives across runs
- **Quality gates as dbt tests** — `unique`, `not_null`, `relationships`, plus custom generic tests `unique_combination_of_columns`, `fan_out_no_surplus`, `value_range_outlier`
- **Rich failure diagnostics** via `store_failures_as: table` — the exact rows that failed a test get persisted and shown in the UI
- **Cross-product references** as real dbt sources — shared conformed dimensions from dependency products show up in the lineage graph

- **AI emits macros natively** — `starSchemaPrompt` tells Claude to produce `{{ source() }}` and `{{ ref() }}` instead of bare table names

The migration was validated end-to-end: on a real product (Articles, 4 tables, Belgian SMB data), both engines produce byte-level equivalent Parquet output (same schema, same 119 + 2922 + 7 + 7 rows, zero rows in either-engine-only set). Four path-resolution bugs were caught during the real-data shadow run and fixed before any production cutover.

Frontend: the Observatory design system

The frontend went through a full visual overhaul: editorial serif display font (Source Serif 4), mono eyebrows (Geist Mono), neutral light surfaces, single accent colour (`ocean` , a muted teal). The `/query` page was split from one 2190-line file into seven focused files. Icon system migrated to `lucide-react` . Legacy CSS classes (`.glass-card` , `.gradient-mesh` , etc.) deleted from `globals.css` once their call-sites were ported.

Underlying principle: editorial typography for content, mono tracked-out uppercase for metadata, a single accent for interactive emphasis. The app looks calm, not busy.